Delty ML Engineer Take-Home Assignment

Healthcare Patient No-Show Prediction

Total Time: 2.5 hours

Download the dataset to get started with your analysis:

Download appointments.csv

Overview

At Delty, we're building AI-powered tools to help healthcare organizations improve patient outcomes. One critical challenge our partners face is **patient no-shows** — when patients miss their scheduled appointments without canceling.

No-shows are costly: they waste provider time, delay care for other patients, and hurt clinic revenue. If we could predict which patients are likely to miss appointments, care teams could proactively reach out to reduce no-shows.

Your task: Build a machine learning system that predicts whether a patient will show up for their scheduled appointment.

The Dataset

We've provided a dataset of ~110,000 medical appointments from a hospital network. Each row represents a single scheduled appointment.

Note: The data may contain some quality issues. Part of the exercise is identifying and handling these appropriately.

Assignment Structure

This assignment has three tiers:

Tier	Requirements
Baseline (Required)	ML pipeline + API + Report
Bonus Level 1	Full-stack scheduling application
Bonus Level 2	Production deployment on GCP/AWS

Baseline Requirements

Part 1: Data Exploration

Explore the dataset and document your findings:

- Basic statistics (distributions, missing values, outliers)
- Target variable distribution (what % are no-shows?)
- Identify any data quality issues
- Initial hypotheses about which features might be predictive

Deliverable: Jupyter notebook section or Colab notebook with visualizations and commentary.

Part 2: Feature Engineering

Define features that you can think will help with the prediction:

1. Explain the intuition (why might this help?)
2. Show how you computed it
3. Validate it adds signal (e.g., correlation with target, feature importance)

We're looking for:

- Creativity and domain thinking
- Clear reasoning for each feature
- Proper implementation (no data leakage!)

Deliverable: Code with comments explaining each feature.

Part 3: Model Training

Train **two models**:

1. **Logistic Regression** — as a simple baseline
2. **Gradient Boosting** (XGBoost, LightGBM, or CatBoost) — as your primary model

For both models:

- Use proper train/test split (recommend 80/20 with stratification)
- Handle categorical variables appropriately
- Document any hyperparameter choices

Deliverable: Training code and saved model files.

Part 4: Evaluation & Report

Evaluate both models and write a half page report covering:

Metrics (compute for both models):

- AUC-ROC
- Precision
- Recall
- F1 Score

Report should answer:

1. Which model performed better? By how much?
2. Why do you think that model performed better?
3. Which features were most important? (show feature importance)
4. What are the tradeoffs between precision and recall for this problem?
5. If the clinic can only make 100 outreach calls per day, how would you use this model to prioritize which patients to call?
6. What are the limitations of your model? What would you improve with more time?

Deliverable: Written report (Markdown, PDF, or in notebook). Aim for short answers in a few sentences.

Part 5: API Endpoint

Create a **FastAPI/Node.js** application that serves predictions from your trained model.

Request body:

```
{
  "gender": "F",
  "age": 45,
```

```
  ...
}
```

Response:

```
{
  "no_show_probability": 0.34,
  "prediction": "Will likely show up",
  "model_used": "lightgbm"
}
```

Bonus Level 1: Full-Stack Application

Build a complete **patient scheduling application** with frontend and backend.

You must use:

- **Frontend:** React + TypeScript
- **Backend:** TypeScript (NestJS is recommended, but alternatives are acceptable)
- **Database:** PostgreSQL (ORM is recommended, but optional)
- **Styling:** Tailwind CSS (component libraries like shadcn/ui, Chakra UI, etc. are optional)
- **Cloud:** Google Cloud Platform or Amazon Web Services

Scenario

You're building a simple appointment scheduling system for a clinic. When a patient books an appointment, the system should:

1. Store the appointment in a database
2. Predict the no-show probability
3. Display insights to help the clinic take action

Frontend Requirements

- **P0: Booking form:** Allow staff to input patient details and schedule an appointment
- **P0: Risk indicator:** After booking, prominently display the no-show probability
- **P2: Appointment list:** Display all scheduled appointments
- **P2: Calendar Feature:** Calendar to show appointments by day/week

---

Bonus Level 2: Production Deployment

Submission Guidelines

Format

1. Upload a short video of the frontend showing the functionalities implemented (only if doing the bonus task) - optional
2. Submit a **GitHub repository**

```
├── README.md                 # Overview, setup instructions, approach
├── notebooks/
│   ├── ..
├── src/
│   ├── ..
├── models/
│   └── model.pkl             # Saved model file(s)
├── frontend/                 # (Bonus) Frontend application
├── backend/                  # (Bonus) Backend application
├── report.md                 # Your written report
├── requirements.txt          # Python dependencies
└── Dockerfile                # (Optional) For containerization
```

Report.md should include:

- Brief description of your approach
- Any assumptions you made
- What you would do differently with more time

How to submit:

Use the **"Submit"** tab above to submit your artifacts. Add a link to your GitHub repo (make sure to give access to **darkshadoww**).

Good luck! We're excited to see your work.
